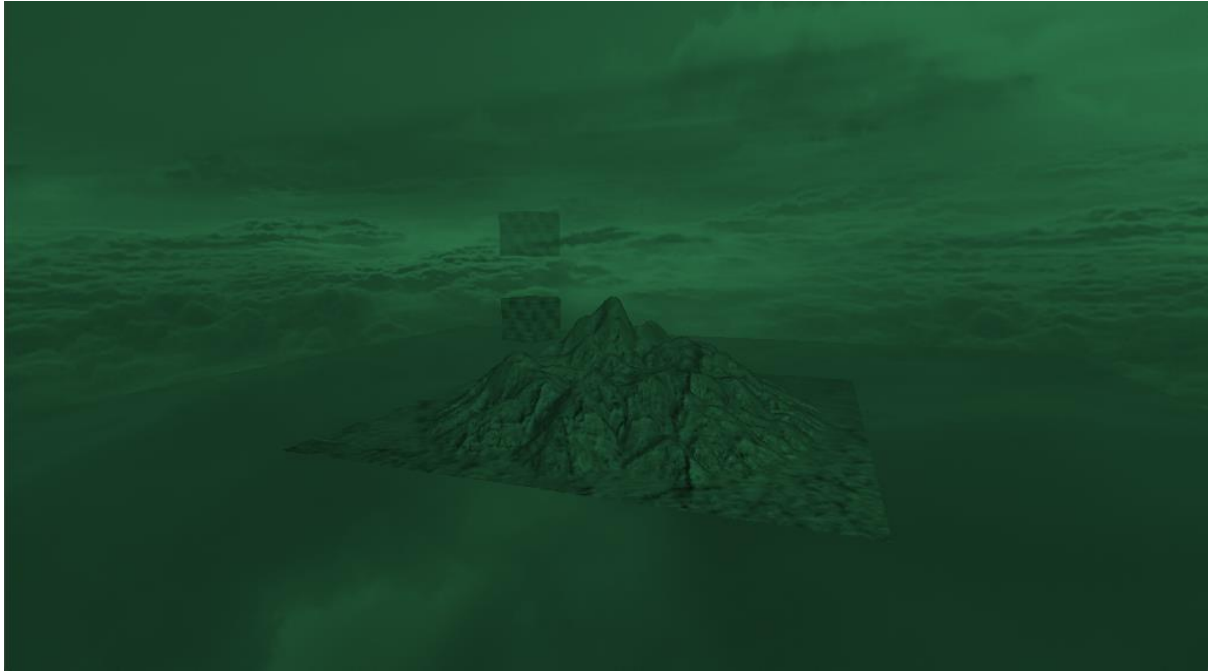


Práctica de Programación Gráfica



Alumno: Samuel Campos Borrego

Entrega: 14-01-2026

Desarrollado en C++ y OpenGL



**UNIVERSIDAD DE DISEÑO,
INNOVACIÓN Y TECNOLOGÍA**

Indice

Indice.....	1
Introducción	2
Clases y funcionamiento de las mismas	3
Scene.....	3
Camera	3
Terrain.....	3
Water	4
Skybox	4
Cube	4
Tree	4
Características Técnicas y Shaders	4
Iluminación	4
Niebla	5
Multitextura	5
Agua Animada	5
Post-Procesado (Framebuffer)	5
EnlaceAl GitHub (Repositorio).....	5

Introducción

Este proyecto consiste en representar un entorno 3D con movimiento mediante una cámara controlada por el usuario con diferentes elementos para visualizar. Como texturas, terrenos, movimientos de los elementos, iluminación, skybox... Todo esto ha sido desarrollado en C++ y OpenGL en VisualStudio 2022.

Clases y funcionamiento de las mismas

Todas las clases contienen un constructor y un destructor además de sus funciones necesarias distribuidas y utilizadas tanto en el .h como en el .cpp.

Scene

Es la clase principal.

- **Responsabilidades:** Inicialización de OpenGL, compilación de shaders, gestión de texturas, actualización de la lógica (Update) y renderizado de todos los elementos (Render).
- **Gestión de Shaders:** Todos los shaders (Terreno, Agua, Cubo, Efectos) están definidos como constantes estáticas (std::string) dentro de esta clase para facilitar su gestión centralizada.
- **Framebuffer:** Gestiona un Frame Buffer para renderizar la escena en una textura antes de mostrarla en pantalla, permitiendo efectos de post-procesado como cambiar el tinte de la cámara.

Camera

- **Movimiento:** Desplazamiento libre en los ejes X, Y, Z con WASD, además de permitir bajar y subir (eje Y) con QE como en Unreal Engine. Con un último añadido de poder moverte más rápido con el Shift (Mayus)
- **Rotación:** Controlada por el ratón (Pitch y Yaw) utilizando matrices de rotación de Euler tras clicar y mantener el clic derecho.
- **Matriz de Vista:** Calcula la matriz LookAt inversa para transformar el mundo relativo a la cámara.

Terrain

- **Geometría:** Una malla plana (GL_LINE_STRIP o triángulos) generada proceduralmente en base a la imagen ubicada en ...\\shared\\assets\\height-map

- **Heightmap:** La forma del terreno no es geometría física, sino que se desplaza visualmente en el Vertex Shader leyendo una textura de mapa de altura (height-map.png). Además de ser afectada por una luz para proyectar sombras y uso de triangulación para conectar los vertex y generar una malla.

Water

- **Geometría:** Un plano simple de dos triángulos.
- **Efecto:** Se renderiza con un shader especial que anima las coordenadas UV y aplica transparencia para simular agua. Además, se modifica su posición para que este a nivel de suelo de la montaña.

Skybox

- **Técnica:** Se renderiza deshabilitando la escritura en el Z-Buffer y eliminando la traslación de la matriz de vista, creando el efecto de que el cielo es "infinito" haciendo que la cámara nunca pueda alcanzarlo, ya que la persigue ubicándose siempre en el 0,0,0 respecto a la cámara.

Cube

- **Soporte:** Admite coordenadas de vértices, coordenadas de textura y normales para iluminación.

Tree

- **Geometría:** Carga un modelo 3D desde un archivo externo .obj, procesando manualmente sus vértices, normales y coordenadas de textura para su representación en la escena.
- **Shaders:** Implementa una técnica de Alpha Testing en el Fragment Shader que descarta los píxeles con baja opacidad, permitiendo renderizar la transparencia de las hojas de forma limpia.
- **Renderizado:** Durante su dibujado se desactiva el Back-face Culling para que las hojas, que son planos bidimensionales, resulten visibles desde cualquier ángulo de la cámara.

Características Técnicas y Shaders

Iluminación

Se ha implementado un modelo de iluminación difusa y ambiental mediante Blinn-Phong (En un inicio se usó Phong, pero se recomendó usar mejor el otro sistema de

iluminación). Para ello se ha creado una clase para la iluminación y ser afectada a todos los objetos de la escena heredando de la clase Node para luego proyectarse en los objetos de la escena.

Niebla

Cálculo: Basado en la distancia del vértice a la cámara

Implementación: Se usa la función mix() en el Fragment Shader para mezclar el color del objeto con el color de fondo (gris) según un factor de visibilidad calculado en el Vertex Shader.

Multitextura

Textura 1: Patrón de números en casillas para saber las dimensiones de la textura en comparación con el objeto (uv-checker.png).

Textura 2: Textura de madera (wood.png).

Mezcla: El shader recibe un array de samplers uniform sampler2D samplers[2] y mezcla ambos colores al 50%.

Agua Animada

Movimiento: Se usa una variable uniforme time que incrementa en cada frame. En el Vertex Shader, se suma este tiempo a las coordenadas UV para desplazar la textura.

Tiling: Las UVs se multiplican por 80.0 para repetir la textura muchas veces, permitiendo que el plano de agua sea gigante sin perder resolución.

Transparencia: Se utiliza glBlendFunc y el canal Alpha para hacer el agua semitransparente.

Post-Procesado (Framebuffer)

Toda la escena se renderiza primero en una textura oculta (out_texture_id). Luego, se dibuja un cuadrado (Quad) que ocupa toda la pantalla con esa textura aplicada.

Efecto Actual: Filtro verdoso. El shader calcula la luminosidad y la multiplica por un vector de color verdoso.

EnlaceAl GitHub (Repositorio)

https://github.com/SamuSamuV/PracticaProgramacionGrafica_SamuelCamposBorrego